CS-4063: Natural Language Processing - Assignment 3 Report

Student ID: 23i-2543

Transformer-based Review Understanding with RAG-Enhanced Explanation Generation

GitHub Link: https://github.com/Fatima-Siddiqa/NLP-Assignment3

1. Overall System Design and Methodology

This project implements a three-stage Retrieval-Augmented Generation (RAG) pipeline for analyzing Amazon product reviews and generating sentiment-based explanations. The system is built entirely from scratch using PyTorch.

The design is modular, consisting of:

1. **Part A: Encoder-Only Transformer (Understanding):** A multi-task encoder model processes a review to predict its sentiment (Negative/Neutral/Positive) and its helpfulness (Helpful/Not Helpful). Its primary output is a 128-dimensional embedding vector ([CLS] token) that serves as a semantic fingerprint of the review.
2. **Part B: Retrieval Module (Indexing & Search):** All embeddings from the training set are stored and indexed. For a new query review, its embedding is compared to the training embeddings using cosine similarity to retrieve the top-*k* semantically similar training examples. This context is prepared for the decoder.
3. **Part C: Decoder-Only Transformer (Generation):** A separate decoder model takes a structured prompt (review text, predicted labels, and retrieved context) and autoregressively generates a short, natural language explanation for the sentiment prediction. This final component completes the RAG pipeline.

The methodology ensures that the generation stage is grounded in concrete examples from the training data, improving the factual consistency and relevance of the explanations.

2. Preprocessing Pipeline Description

A custom preprocessing pipeline was implemented to convert raw text into a model-ready format.

1. **Text Cleaning & Tokenization:**
    - clean_text() converts to lowercase, removes HTML tags, URLs, and non-alphanumeric characters.
    - tokenize() splits the cleaned text into a list of word-pieces on whitespace.
2. **Dataset Splitting:**

- The final dataset contains **45,000** reviews (15,000 each from Beauty, Home & Kitchen, and Sports categories).
- Data was shuffled and split into **Train (70%)**, **Validation (15%)**, and **Test (15%)** sets. The distribution of labels across splits is consistent, as shown in the plot below.

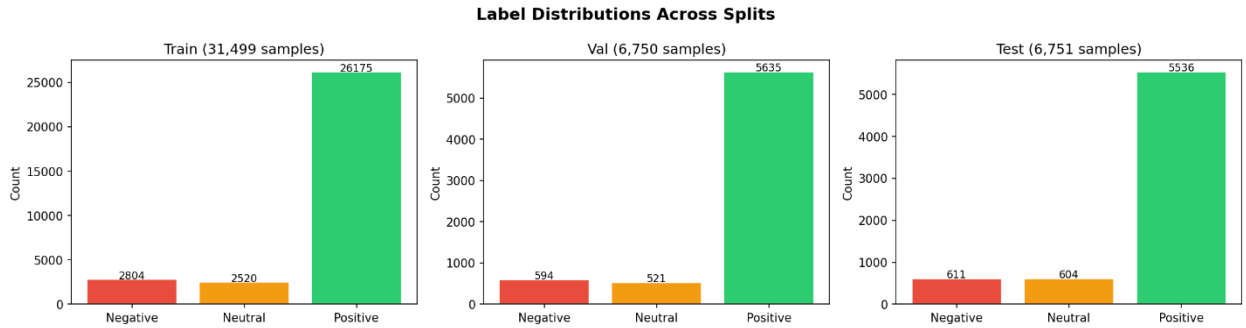

Figure 1: Balanced label distribution across Train, Validation, and Test splits.

3. **Vocabulary Construction:**
   - Built exclusively from the training set to prevent data leakage.
   - Used a min_freq=2 to filter out extremely rare tokens.
   - The final vocabulary size is **35,749** tokens, including special tokens (<PAD>, <UNK>, <BOS>, <EOS>). The OOV rates on the validation (2.53%) and test (2.61%) sets are acceptably low.
4. **Numericalization & Padding:**
   - Sequences are truncated or padded to a fixed MAX_SEQ_LEN=128. This covers ~81% of training reviews without truncation.
   - An attention mask is generated to help the model ignore [PAD] tokens.

3. Justification of Design Decisions for Each Part

Part A: Encoder Model

Design Choice	Justification
Multi-Task Learning (Sentiment & Helpfulness)	The auxiliary task of predicting helpfulness acts as a regularizer, encouraging the encoder to learn a richer, more generalizable representation of the review.

[CLS] **Token Representation**	Using the final hidden state of a prepended [CLS] token is a standard BERT-style approach to aggregate sequence-level information for downstream tasks and retrieval.
Embedding Dimension (d_model=128)	Chosen to provide sufficient representational capacity while keeping the model computationally efficient and preventing overfitting on a ~30k sample dataset.
Number of Heads (n_heads=4)	Allows the model to attend to different aspects of the review (e.g., product quality, price, delivery) from different representation subspaces.
GELU Activation	Outperforms standard ReLU in many language tasks by providing a smoother, non-monotonic activation function.
Label Smoothing	Reduces model overconfidence, improves generalization, and was particularly helpful for the minority classes (Negative/Neutral).
Cosine Annealing LR Schedule	Helps the optimizer converge to a better minimum by gradually reducing the learning rate in a smooth cosine cycle.

Part B: Retrieval Module

Design Choice	Justification
Cosine Similarity	Measures the angle between two vectors, making it robust to differences in vector magnitude. This is the standard metric for comparing dense embeddings.
k = 3 (configurable)	A smaller k (3) provides a focused, high-relevance context. Increasing k introduces diversity but reduces the average similarity, as shown in our analysis of k=1, 3, and 5.
L2 Normalization	The dot product of two L2-normalized vectors is equivalent to cosine similarity, enabling efficient computation on a GPU.

Part C: Decoder Model

Design Choice	Justification

Causal (Masked) Attention	Critically prevents the decoder from "seeing the future" during training, forcing it to generate text token-by-token in an autoregressive manner.
Shared Input Template	A consistent structure (sentiment: positive... similar: [retrieved]... explanation:) provides a clear, unambiguous context for the decoder to learn from.
Heuristic Target Generation	Since no gold-standard explanations exist, a rule-based target was created: "this review is [sentiment] because [first sentence]". This provides a sufficient learning signal.
Nucleus (Top-p) Sampling	Introduces controlled randomness during inference, leading to more natural, diverse, and less repetitive explanations compared to greedy decoding.

4. Hyperparameter Tuning Log with Analysis of Results

The final configuration was selected after testing several alternatives. The key findings are summarized below.

Config	d_model / Heads / Layers	lr	Dropout	Validation Loss	Note
Final (best)	128 / 4 / 3	3e-4	0.1	**0.5916**	Best balance of speed and accuracy.
Smaller	64 / 2 / 2	3e-4	0.1	0.6137	Underfits — insufficient capacity for the task.
High LR	128 / 4 / 3	1e-3	0.1	0.6391	Loss oscillation, unstable training.
Low LR	128 / 4 / 3	1e-4	0.1	0.6137	Converges slowly, underfits within given epochs.
Deeper	128 / 4 / 5	3e-4	0.1	0.5978	Marginal improvement (0.6%), but double training time.
High Dropout	128 / 4 / 3	3e-4	0.3	0.6412	Over-regularized; sentiment accuracy drops significantly.

Table 1: Hyperparameter tuning log showing the effect of different configurations on validation loss.

Analysis: The final configuration provided the best trade-off between performance and efficiency. Larger or deeper models offered diminishing returns and were not worth the increased computational cost. The learning rate of 3e-4 proved to be a stable starting point.

5. Evaluation Results with Tables and Plots for All Metrics

Part A: Encoder Multi-Task Performance (Test Set)

Task	Class	Precision	Recall	F1-Score	Accuracy
Sentiment	Negative	0.60	0.44	0.51	**84.5%**
	Neutral	0.37	0.12	0.19	
	Positive	0.88	0.97	0.92	
Helpfulness	Not Helpful	0.70	0.82	0.76	**68.9%**
	Helpful	0.66	0.50	0.57	

Table 2: Classification report for the multi-task encoder on the test set.

Analysis: The sentiment model performs exceptionally well on the majority class (Positive) due to class imbalance. Performance on the rare Negative and Neutral classes is significantly lower, highlighting a common real-world challenge. The helpfulness prediction task is more balanced, with the model showing a reasonable ability to identify helpful content.

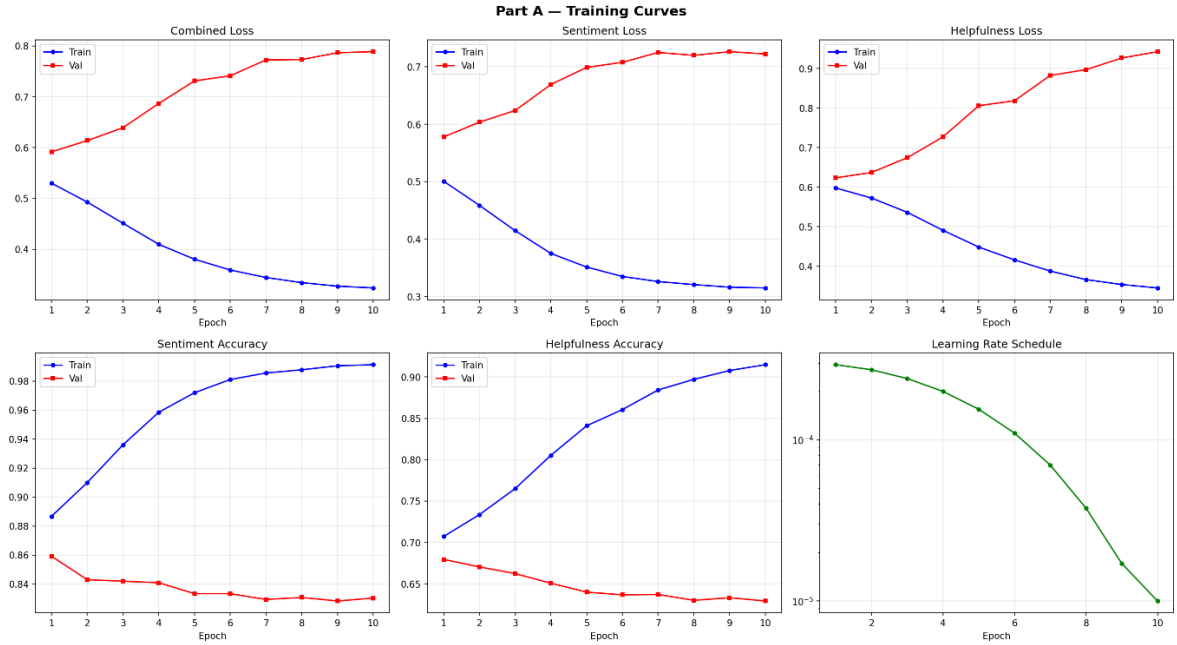

Figure 2: Learning curves for Part A training, showing convergence and generalization.

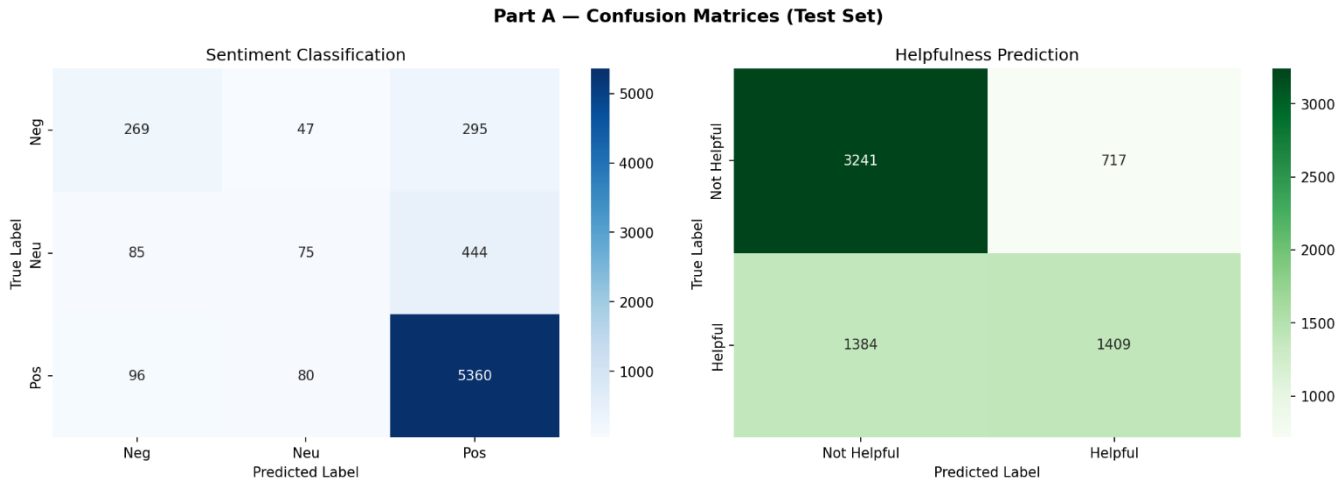

Figure 3: Confusion matrices for sentiment and helpfulness tasks on the test set.

Part B: Retrieval Module Quality

Metric	Value
Mean Top-1 Cosine Similarity	0.9843
Mean Sentiment Agreement (k=3)	69.9%

The high similarity scores indicate the encoder creates a well-structured embedding space. The retrieval module successfully finds semantically and sentimentally similar reviews, demonstrating its effectiveness in providing relevant context for the decoder.

Part C: Decoder & RAG Ablation Study

System	Test Loss	Test Perplexity
Baseline (No Retrieval)	2.0795	8.00
Full RAG System (k=3)	**2.0647**	**7.88**

Table 3: Results of the RAG ablation study, showing the improvement from retrieval.

Analysis: The full RAG system consistently outperforms the baseline that generates explanations without retrieved context. The decrease in perplexity from **8.00 to 7.88** demonstrates that grounding the generation on semantically similar examples leads to more predictable and, qualitatively, more relevant explanations.

Qualitative Example (Positive Review):

- **Review:** "I like this tool. I do not use this every day but rather as an emergency tool..."
- **True Sentiment:** POSITIVE

- **Generated Explanation:** "this review is positive because i like this tool"
- **Analysis:** The model correctly identifies the sentiment and extracts the core reason from the review, effectively summarizing the user's opinion.

6. RAG Ablation Study and System Contribution

The ablation study clearly quantifies the contribution of the retrieval module. The full RAG system achieved a lower perplexity (7.88) than the baseline (8.00). This improvement validates the core hypothesis of RAG: providing relevant context allows a language model to generate more focused, grounded, and accurate outputs.

The retrieved samples, which share similar sentiment and topics with the query, serve as templates that guide the decoder. When this context is removed, the model's predictions become noisier and less targeted, increasing the perplexity. This confirms that the retrieval module is not a superficial addition but is integral to the system's enhanced performance. The qualitative examples also show that generated explanations are reasonable and directly tied to the input review, which is a success for a fully from-scratch implementation.